TRAVAIL NOTÉ

Travail noté 1

- Remplissez soigneusement cette feuille d'identité.
- Rédigez votre travail, en commençant à la page suivante.
- Sauvegardez votre travail de cette façon : SIGLEDUCOURS_TN1_VOTRENOM.
- Utilisez le *Dépôt des travaux* pour acheminer votre travail à votre professeur ou son délégué. http://www.teluq.ca/mateluq/

Feuille d'identité

Nom Lépine Prénom Alexandra

Numéro d'étudiant 24122245 Trimestre Hiver 2026

Adresse 271 rue Jeannotte

Mont-St-Hilaire Code postal J3H 5G2

Téléphone Domicile _____ Travail _____

Cellulaire 450-209-0417

Courriel lepine.alexandra@univ.teluq.ca

Nom du professeur ou son délégué Christian Jauvin

Réservé à l'usage du professeur ou son délégué

Date de réception _____ Date de retour _____

Note _____

Date d'envoi _____

Premier problème :
Produisez un pseudo-code précis et expliquez votre solution. Vous pouvez expliquer le code sous la forme que vous souhaitez, tant que vos explications sont claires. Exécutez votre pseudo-code sur au moins 5 cas, incluant les deux cas suivants: le tableau [55,55,55,55] et le tableau [1,2,3,5,6].

L'algorithme doit déterminer si la somme des éléments d'un tableau de nombres positif excède 100. De plus, l'algorithme doit être efficace, l'algorithme doit accéder è aussi peu d'éléments possible du tableau. Il faudra exécuter le pseudo-code 5 fois en incluant les cas suivants : le tableau [55,55,55,55] et le tableau [1,2,3,5,6].

Selon l'indice 2, l'algorithme doit retourner soit vrai, soit faux.
Selon l'indice 3,l'algorithme doit fonctionner même si le tableau à une longueur de zéro.
Selon l'indice 4, l'algorithme doit fonctionner même si le tableau contient mille milliards d'élément

Suite à l'indice 1, dans claude.ia j'ai saisie l'énoncé :
Nous souhaitons un algorithme qui détermine si la somme des éléments d'un tableau de nombres positifs excède 100. Par ailleurs, nous souhaitons un algorithme efficace: l'algorithme doit accéder à aussi peu d'éléments du tableau que possible. Produisez un pseudo-code précis et expliquez votre solution. Vous pouvez expliquer le code sous la forme que vous souhaitez, tant que vos explications sont claires. Exécutez votre pseudo-code sur au moins 5 cas, incluant les deux cas suivants: le tableau [55,55,55,55] et le tableau [1,2,3,5,6].

Dans la version de claude.ia (Sonnet 4.6), du 19 février 2026, j'ai obtenu le résultat suivant :

Idée clé
Puisque tous les éléments sont positifs, dès que la somme partielle dépasse 100, on peut s'arrêter immédiatement — les éléments suivants ne peuvent qu'augmenter la somme. Il est donc inutile de parcourir tout le tableau.

Pseudo-code

```
fonction excède100(T) :
  somme ← 0
  pour chaque élément x dans T :
    somme ← somme + x
    si somme > 100 :
      retourner VRAI
  retourner FAUX
```

Explication

L'algorithme accumule les éléments un à un dans une variable somme. Après chaque addition, il vérifie si le seuil est franchi. Si c'est le cas, il retourne VRAI sans toucher aux éléments restants. Si on parcourt tout le tableau sans jamais dépasser 100, on retourne FAUX.
Pourquoi c'est optimal ? On ne peut pas faire moins d'accès : dans le pire cas (réponse FAUX), il faut nécessairement lire tous les éléments pour être certain que la somme ne dépasse pas 100. Dans le meilleur cas (réponse VRAI), on s'arrête au plus tôt possible.

Exécutions

Cas 1 — [55, 55, 55, 55]

Étape	x	somme	> 100 ?
1	55	55	Non
2	55	110	**Oui** → retourner VRAI

Éléments accédés : **2 sur 4**. Résultat : **VRAI**

Cas 2 — [1, 2, 3, 5, 6]

Étape	x	somme	> 100 ?
1	1	1	Non
2	2	3	Non
3	3	6	Non
4	5	11	Non
5	6	17	Non

Fin du tableau. Résultat : **FAUX**

Cas 3 — [200]

Étape	x	somme	> 100 ?
1	200	200	**Oui** → retourner VRAI

Éléments accédés : **1 sur 1**. Résultat : **VRAI**

Cas 4 — [10, 20, 30, 40]

Étape	x	somme	> 100 ?
1	10	10	Non
2	20	30	Non
3	30	60	Non
4	40	100	Non (100 n'est pas **strictement** supérieur à 100)

Fin du tableau. Résultat : **FAUX**

Cas 5 — [] (tableau vide)

La boucle ne s'exécute jamais. somme reste à 0. Résultat : **FAUX**

Cas 6 — [50, 49, 1, 1, 1]

Étape	x	somme	> 100 ?
1	50	50	Non
2	49	99	Non
3	1	100	Non
4	1	101	**Oui** → retourner VRAI

Éléments accédés : **4 sur 5**. Résultat : **VRAI**

Réalisation du pseudo-code

J'ai décidé de débuter mon pseudo-code à partir des informations fournit à l'indice 6. Dans le pseudo-code, j'ai mis une condition d'arrêt anticipé afin que l'algorithme retourne soit vrai, soit faux. Pour ce qui est de la condition que l'algorithme doit fonctionner même si le tableau a une longueur de zéro, la condition iterateur < longueur de tableau est évaluée avant d'entrée de la boucle donc si le tableau est vide, la condition retourne automatiquement à faux.

```
Entrée :
Tableau de nombres positifs : tableau

Variables :
Nombre entier : iterateur = 0;

Sorties :
Nombre : somme = 0;
Booléen : résultat #le résultat est soit vrai ou faux

TANT QUE iterateur < la longueur de tableau FAIRE
     somme = somme + tableau [iterateur];
     iterateur = iterateur + 1;
     SI somme > 100 ALORS
          résultat = VRAI
          ARRÊTER
     FIN SI
FIN TANT QUE

résultat = FAUX
```

Exécution du pseudo-code

Tableau [55,55,55,55]

Itération 1

```
TANT QUE iterateur 0 <4
     Somme = 0 + 55 = 55
     iterateur = 0 + 1 = 1
```

```
     SI 55 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 2

```
TANT QUE iterateur 1 < 4
     Somme = 55 + 55 = 110
     iterateur = 1+ 1 = 2
     SI 110 < 100 // 110 est supérieur à 100, donc
l'algorithme va arrêter
     résultat = VRAI
     ARRÊTER
FIN SI
```

Tableau [1,2,3,5,6]

Itération 1

```
TANT QUE iterateur 0 < 5
     somme = 0 + 1 = 1
     iterateur = 0 + 1 = 1
     SI 1 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 2

```
TANT QUE iterateur 1< 5
     somme = 1 + 2 = 3
     iterateur = 1 + 1 = 2
     SI 3 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 3

```
TANT QUE iterateur 2< 5
     somme = 3 + 3 = 6
     iterateur = 2 + 1 = 3
     SI 6 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 4

```
TANT QUE iterateur 3< 5
     somme = 6 + 5 = 11
     iterateur = 3 + 1 = 4
     SI 11 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 5

```
TANT QUE iterateur 4< 5
     somme = 11 + 6 = 17
     iterateur = 4 + 1 = 5
     SI 17 > 100 ALORS // Non, on continue l'algorithme
```

```
FIN SI
```

Itération 6

```
TANT QUE iterateur 5< 5 // Faux, on sort de la boucle
     résultat = FAUX
```

Tableau [12,25,31,45]

Itération 1

```
TANT QUE iterateur 0 < 4
     somme = 0 + 12 = 12
     iterateur = 0 + 1 = 1
     SI 12 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 2

```
TANT QUE iterateur 1 < 4
     somme = 12 + 25 = 37
     iterateur = 1 + 1 = 2
     SI 37 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 3

```
TANT QUE iterateur 2 < 4
     somme = 37 + 31 = 68
     iterateur = 2 + 1 = 3
     SI 68 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 4

```
TANT QUE iterateur 3 < 4
     Somme = 68+ 45 = 113
     iterateur = 3 + 1 = 4
     SI 113 > 100 // 113 est supérieur à 100, donc
l'algorithme va arrêter
     résultat = VRAI
     ARRÊTER
FIN SI
```

Tableau[15,65,70,97]

Itération 1

```
TANT QUE iterateur 0 < 4
     somme = 0 + 15 = 15
     iterateur = 0 + 1 = 1
     SI 15 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 2

```
TANT QUE iterateur 1 < 4
     somme = 15 + 65 = 80
     iterateur = 1 + 1 = 2
     SI 80 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 3

```
TANT QUE iterateur 2 < 4
     Somme = 80 + 70 = 150
     iterateur = 2 + 1 = 3
     SI 150 > 100 // 150 est supérieur à 100, donc
l'algorithme va arrêter
     résultat = VRAI
     ARRÊTER
FIN SI
```

Tableau [1,5,10,15,16,17]

Itération 1

```
TANT QUE iterateur 0 < 6
     somme = 0 + 1 = 1
     iterateur = 0 + 1 = 1
     SI 1 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 2

```
TANT QUE iterateur 1 < 6
     somme = 1 + 5 = 6
     iterateur = 1 + 1 = 2
     SI 6 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 3

```
TANT QUE iterateur 2 < 6
     somme = 6 + 10 = 16
     iterateur = 2 + 1 = 3
     SI 16 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 4

```
TANT QUE iterateur 3 < 6
     somme = 16 + 15 = 31
     iterateur = 3 + 1 = 4
     SI 31 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 5

```
TANT QUE iterateur 4 < 6
     somme = 31 + 16 = 47
```

```
     iterateur = 4 + 1 = 5
     SI 47 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 6

```
TANT QUE iterateur 5 < 6
     somme = 47 + 17 = 64
     iterateur = 5 + 1 = 6
     SI 64 > 100 ALORS // Non, on continue l'algorithme
FIN SI
```

Itération 7

```
TANT QUE iterateur 6< 6 // Faux, on sort de la boucle
     résultat = FAUX
```

Interprétation des résultats

<u>Tableau [55,55,55,55]</u>
Dans le tableau ayant comme données [55,55,55,55], la somme a dépassé 100 dès la deuxième itération. Le pseudo-code s'est arrêté immédiatement sans parcourir tout le tableau. Cela signifie que la somme des éléments dépasse 100.

<u>Tableau [1,2,3,5,6]</u>
Dans le tableau ayant comme données [1,2,3,5,6], le pseudo-code a parcouru toutes les itérations sans dépasser la somme de 100. La somme finale est de 17, ce qui est en dessous de 100. Cela signifie que la somme des éléments ne dépasse pas 100.

<u>Tableau [12,25,31,45]</u>
Dans le tableau ayant comme données [12,25,31,45], la somme a dépassé 100 lors de la quatrième itération. Le pseudo-code s'est arrêté immédiatement. Cela signifie que la somme des éléments dépasse 100.

<u>Tableau [15,65,70,97]</u>
Dans le tableau ayant comme données [15,65,70,97], la somme a dépassé 100 lors de la troisième itération. Le pseudo-code s'est arrêté immédiatement sans parcourir tout le tableau. Cela signifie que la sommes des éléments dépasse 100.

<u>Tableau [1,5,10,15,16,17]</u>
Dans le tableau ayant comme données [1,5,10,15,16,17], le pseudo-code a parcouru toutes les itérations sans dépasser la somme de 100. La somme finale est de 64, ce qui est en dessous de 100. Cela signifie que la somme des éléments ne dépasse pas 100.

Second problème
Expliquer l’erreur et offrez une version corrigée. Exécutez le pseudo-code corrigé, ainsi que le pseudo-code original. Expliquer les différences.

Il faut corriger le pseudo-code fournit par un étudiant, l’erreur est logique et non esthétique. Le pseudo-code doit afficher les nombres entiers de 0 à 100, incluant 0 mais excluant 100, en affichant « Fizz » quand le nombre est divisible par 3 et « Buzz » quand le nombre est divisible par 5.

Suite à l’indice, dans claude.ia j’ai saisie :
Nous souhaitons afficher les nombres entiers de 0 jusqu'à 100 à l'écran (incluant 0 mais excluant 100), en affichant "Fizz" quand le nombre est divisible par 3 et "Buzz" quand le nombre est divisible par 5. Expliquez l'erreur dans ce pseudocode: TANT QUE iterateur < 100 FAIRE affiche la valeur de iterateur à l'écran SI iterateur est divisible par 3 ALORS affiche " Fizz" à l'écran SINON SI iterateur est divisible par 5 ALORS affiche " Buzz" à l'écran FIN SI change de ligne à l'écran iterateur = iterateur + 1 FIN TANT QUE ».

Dans la version de claude.ia (Sonnet 4.6), du 20 février 2026, j’ai obtenu le résultat suivant :

Il y a deux erreurs dans ce pseudocode :
1. L'itérateur n'est jamais initialisé
Avant la boucle, il manque iterateur = 0. Sans cette initialisation, la valeur de départ est indéfinie.
2. La structure conditionnelle utilise SINON SI au lieu de deux SI indépendants
C'est l'erreur principale de logique. Avec SINON SI, les deux conditions sont mutuellement exclusives : si le nombre est divisible par 3, on affiche "Fizz" et on ne vérifiera **jamais** s'il est aussi divisible par 5. Résultat : le nombre 15 (divisible par 3 et par 5) affichera uniquement "Fizz", alors qu'il devrait afficher "Fizz Buzz".
Il faudrait remplacer le SINON SI par un SI indépendant :
SI iterateur est divisible par 3 ALORS
  affiche " Fizz" à l'écran
FIN SI
SI iterateur est divisible par 5 ALORS
  affiche " Buzz" à l'écran
FIN SI
Ainsi les deux conditions sont évaluées séparément pour chaque nombre.

Pseudo-code contenant l’erreur :
```
Entrées :
Valeur maximale :  100;

Variables :
Nombre entier : iterateur = 0;
```

```
Sorties : à l'écran


TANT QUE iterateur < 100 FAIRE
       affiche la valeur de iterateur à l'écran
       SI iterateur est divisible par 3 ALORS
         affiche " Fizz"  à l'écran
       SINON SI iterateur est divisible par 5 ALORS
         affiche " Buzz"  à l'écran
       FIN SI
       change de ligne à l'écran
       iterateur = iterateur + 1
FIN TANT QUE
```

Réalisation du pseudo-code sans erreur :

```
Entrées :
Valeur maximale : 100

Variables :
Nombre entier : iterateur 0;

Sorties : à l’écran

Tant que iterateur < 100 FAIRE
     affiche la valeur de iterateur à l’écran
     SI iterateur est divisible par 3 ALORS
          affiche ‘’Fizz’’
     FIN SI
     SI iterateur est divisible par 5 ALORS
          affiche ‘’Buzz’’ à l’écran
     FIN SI
     change de ligne à l’écran
     iterateur = iterateur + 1
FIN TANT QUE
```

Exécution du pseudo-code non corrigé :
Voici un résumé des itérations du pseudo-code non corrigé

Itération 1

```
TANT QUE iterateur 0 <100
     affiche 0
     SI 0 est divisible par 3 //oui
          affiche ‘’Fizz’’
     SINON SI 0 divisible par 5//non vérifié car vrai pour 3
     change de ligne à l’écran
```

```
     iterateur = 0+1 = 1
FIN SI
```

Itération 2

```
TANT QUE iterateur 1 <100
     affiche 1
     SI 1 est divisible par 3 //non

     SINON SI 1 divisible par 5 // non
     change de ligne à l'écran
     iterateur = 1+1 = 2
FIN SI
```

Itération 6

```
TANT QUE iterateur 5 <100
     affiche 5
     SI 5 est divisible par 3 //non

     SINON SI 5 divisible par 5 // oui
          affiche ''Buzz''
     change de ligne à l'écran
     iterateur = 5+1 = 6
FIN SI
```

Itération 16

```
TANT QUE iterateur 15 <100
     affiche 15
     SI 1 est divisible par 3 //oui
          affiche ''Fizz''
     SINON SI divisible par 5 // non vérifié car vrai pour 3
     change de ligne à l'écran
     iterateur = 15+1 = 16
FIN SI
```

Itération 16

```
TANT QUE iterateur 30 <100
     affiche 30
     SI 1 est divisible par 3 //oui
          affiche ''Fizz''
     SINON SI divisible par 5 // non vérifié car vrai pour 3
     change de ligne à l'écran
     iterateur = 30+1 = 31
FIN SI
```

Exécution du pseudo-code corrigé :

Voici un résumé des itérations du pseudo-code corrigé

Itération 1

```
TANT QUE iterateur 0 <100
```

```
     affiche 0
     SI 0 est divisible par 3 // oui
          affiche ''Fizz''
     SI 0 divisible par 5 // oui
          affiche ''Buzz''
     change de ligne à l'écran
     iterateur = 0+1 = 1
```

Itération 2

```
TANT QUE iterateur 1 <100
     affiche 1
     SI 1 est divisible par 3 //non
     SI 0 divisible par 5 // non
     change de ligne à l'écran
     iterateur = 1+1 = 2
```

Itération 6

```
TANT QUE iterateur 5 <100
     affiche 5
     SI 5 est divisible par 3 // non
     SI 0 divisible par 5 // oui
          affiche ''Buzz''
     change de ligne à l'écran
     iterateur = 5+1 = 6
```

Itération 16

```
TANT QUE iterateur 15 <100
     affiche 15
     SI 0 est divisible par 3 // oui
          affiche ''Fizz''
     SI 0 divisible par 5 // oui
          affiche ''Buzz''
     change de ligne à l'écran
     iterateur = 15+1 = 16
```

Itération 31

```
TANT QUE iterateur 30 <100
     affiche 30
     SI 0 est divisible par 3 // oui
          affiche ''Fizz''
     SI 0 divisible par 5 // oui
          affiche ''Buzz''
     change de ligne à l'écran
     iterateur = 30+1 = 31
```

Explication de la différence entre les deux pseudo-code :

La différence qu'il y a entre le pseudo-code non corrigé et le pseudo-code corrigé est la structure conditionnelle. Dans le premier algorithme, la structure conditionnelle est « SINON SI » comparativement au second algorithme que la structure conditionnelle est « SI ». Avec la première structure conditionnelle, les deux conditions sont mutuellement exclusives, donc si l'itérateur est divisible par 3 le pseudo-code ne va pas valider si le nombre est divisible par 5.La structure conditionnelle « SINON SI » fait en sorte que le nombre 15 a comme résultat "Fizz" et que le nombre 30 a comme résultat « « Fizz ». Avec la seconde structure conditionnelle « SI » les conditions sont séparées, ce qui fait en sorte qu'elles sont indépendantes, chaque condition est toujours évaluée, peu importe le résultat de l'autre. La structure condition « SI » fait en sorte que le nombre 15 a comme résultat « Fizz Buzz » et que le nombre 30 à comme résultat « Fizz Buzz ».